Task 4: Electronics Store Inventory

BCS | Java | 40 minutes | Instructor: Dr Muhammad Shahid Bhatti

Build a store system and test behavior with arrays and overriding.

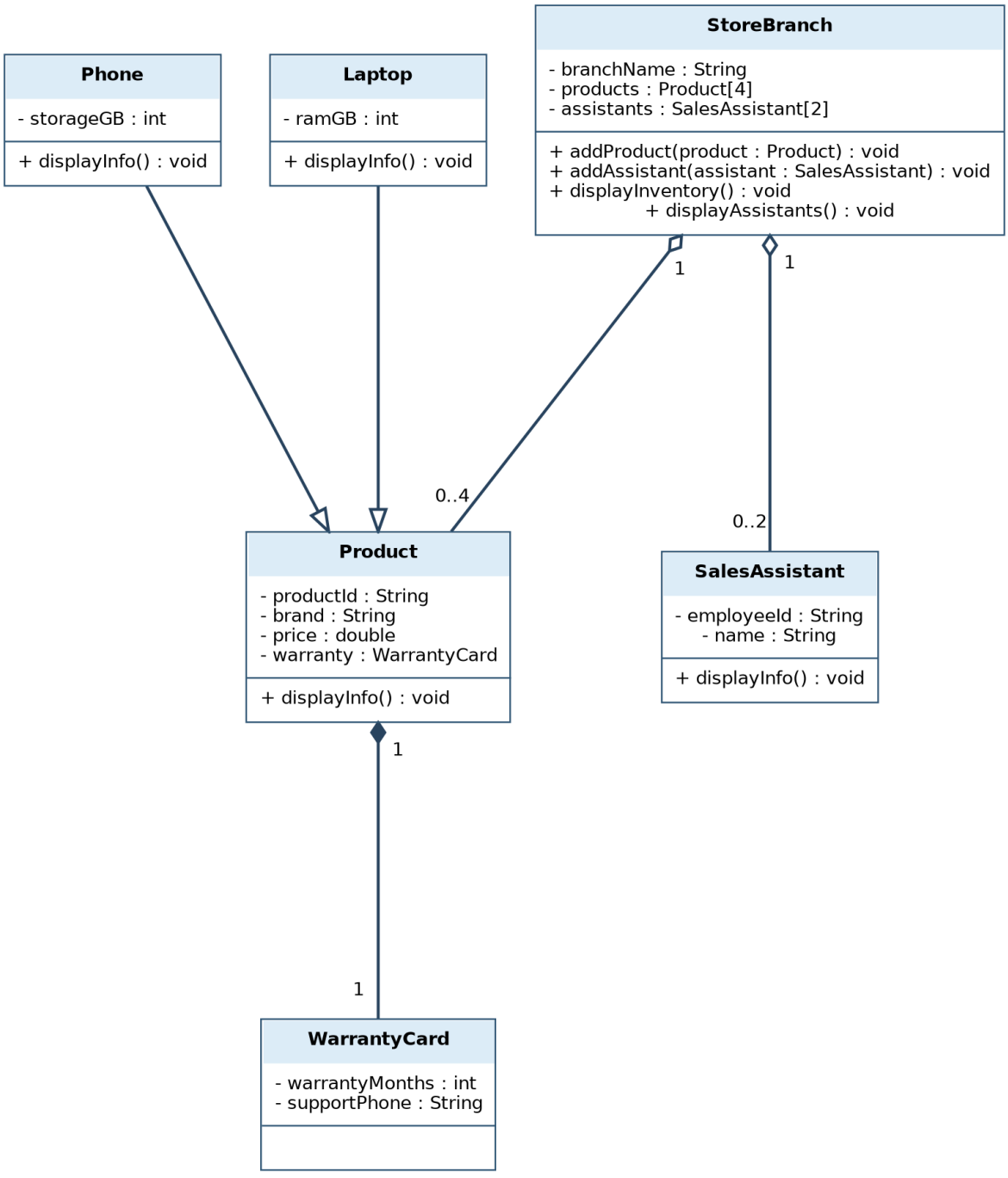
Scenario

A neighborhood electronics shop sells phones and laptops. Each Product has a product ID, brand, and price. A Phone adds storage capacity in GB; a Laptop adds RAM size in GB. Displaying each type of product should include its special property.

Each Product has a WarrantyCard recording warranty months and a support phone number. The card is created within the Product constructor using values supplied for that particular product. Do not create a WarrantyCard separately in main.

A StoreBranch has a branch name. It can display up to four already-created Product objects and up to two already-created SalesAssistant objects. Each SalesAssistant has an employee ID and name. The store does not create products or sales assistants; they are passed to its add methods.

UML Class Diagram



Read the UML notation: hollow triangle = inheritance; black diamond = composition; white diamond = aggregation; plain line = association. Multiplicities show how many objects participate. The editable PlantUML source is provided separately.

Classes and Attributes

Class	Required attributes
Product	productId, brand, price, WarrantyCard warranty
Phone extends Product	storageGB
Laptop extends Product	ramGB
WarrantyCard	warrantyMonths, supportPhone

SalesAssistant	employeeId, name
StoreBranch	branchName, Product[] products, SalesAssistant[] assistants

Required Methods

- Product.displayInfo(); override it in Phone and Laptop using super.displayInfo().
- StoreBranch.addProduct(Product product) and addAssistant(SalesAssistant assistant).
- StoreBranch.displayInventory(): display the existing products.
- StoreBranch.displayAssistants(): show assigned staff.

What to Create in main()

- Create two Phone objects, one Laptop object, and two SalesAssistant objects.
- Create a StoreBranch; add all existing products and assistants.
- Display inventory and staff details.
- Attempt to add a fifth product and print a meaningful full-capacity warning.

Relationships to Demonstrate

Inheritance: Phone and Laptop extend Product. Composition: Product creates and owns WarrantyCard. Aggregation: StoreBranch keeps existing Product and SalesAssistant references.

Completion Checklist

- ☐ All classes compile and run.
- ☐ Constructors use super(...) in subclasses.
- ☐ Inherited display methods are overridden where specified.
- ☐ Composition objects are constructed within their owner.
- ☐ Aggregation objects are created first and added/passed afterward.
- ☐ The program runs and prints understandable output.

Submission

Save all .java source files for this task in its own folder. At the end of the lab, submit ONE ZIP file containing completed folders for Tasks 1–4. Do not submit four separate ZIP files.